
REPORT · 2026-09-30

Alberto AI v1.8.1 — Capacidades Reais

O que funciona, o que não, e a verdade sobre AIOX

AUTHOR

DATE

2026-09-30

✓ O que Alberto CONSEGUE fazer (verificado, real)

1. 💬 Chat natural em PT-BR / EN

Como: 16 workflows × 80 steps × 37 personas **Testes:** `pytest tests/test_v18_workflows.py` (147 testes) **E2E real:** já vimos funcionar com diffusiongemma respondendo em PT-BR

Exemplos de prompts que funcionam:

- "Ola Alberto, me explique quantum computing"
- "Quem foi Agostinho Neto?"
- "Me ajude a escrever um email profissional"

2. 🧠 Multi-persona workflows (squads)

Como: 15 workflows YAML reais executando 1 chain de LLM calls **Engine:** `alberto.agent.run_squad()` chama `model_router.invoke()` para cada step **Latência:** ~5-30s para workflow completo (depende de LLM)

Os 15 squads disponíveis agora:

Workflow	Domain	Steps
engineering	Spec → Arch → Code → QA → Deploy	5
content	Research → Outline → Draft → Edit → Polish	5
finance	Frame → Model → Critique → Decide → Recommend	5
incident	Triage → Diagnose → Communicate → Post-mortem	5
support	Reproduce → Classify → Fix → Verify → Close	5
sales	Qualify → Engineer review → Architect → Finance → Close	5
hiring	JD → Source → Phone screen → Tech interview → Decide → Offer	6
mentorship	1:1 → Skill gap → Pair → Reflect → Career path → Next 90	6
sprint-planning	Agenda → Goal → Estimate → Capacity → Test plan → Summary	6
security-audit	Scope → Scan → Threat model → Remediate → Verify → Report	6
data-ml	Metrics → Pipeline → Train → Deploy → Monitor	5
ml-ops	Monitor → Data health → Retrain → Deploy → Verify	5
growth	Funnel → Rank levers → Hypothesis → Variants → Experiment → Readout	6
product	UX research → PM → Design → Analytics → Tech lead	5
research	Research → Review → Analyze → Write → Critique	5

3. Vision (analisa imagens)

Como: `tool_vision` chama modelo multimodal (diffusionngemma, kimi-k3) **Testado:** já viu "caminho de madeira" em PT-BR **Limitação:** 2 modelos específicos funcionam; llama-vision e phi-vision falham

4. Web search SEM API key (v1.8)

Como: Playwright + Chromium (já instalado) → DuckDuckGo HTML **Engine:**

`_try_browser_search()` em `hermes_tools.py` **Validado:** queries em PT e EN retornam resultados reais

```
>>> _try_browser_search("PIB Angola 2024")
['INE-Instituto Nacional De Estatísticas', ...]
```

5. 🖼️ Image generation (flux.1-dev)

Como: NVIDIA API `https://ai.api.nvidia.com/v1/genai/black-forest-labs/flux.1-dev`

Validado: 5 imagens reais geradas (Rio, Angola, Africa, etc) **Limitação:** só funciona com chave Qwen/flux específica; chaves padrão retornam 404

6. 📁 Memory (FTS5 full-text search)

Como: `tool_memory_set/get/list/search` — SQLite FTS5 **Persistência:** `~/.alberto/memory.db`

Testes: 11 testes em `test_v15_unit.py::TestMemoryOperations`

7. 📝 Code execution (sandboxed)

Como: `tool_code_execution` + `LocalSandbox` (UUID-isolated) **Engine:** Subprocess Python dentro de `/tmp/alberto-sandbox/` **Segurança:** shell whitelist (10+ blocked patterns em v1.7)

8. 🔒 Security scanning (NemoClaw)

Como: 36 secret patterns + 18 protected paths **Engine:** `alberto/runtime/nemoclaw_real.py`

Testes: 21 regression tests em `test_v17_security.py`

Detecta:

- AWS, GitHub, Slack, Stripe tokens
- OpenAI keys, private keys
- `/etc/` , `/root/.ssh/` , `~/.aws/` etc

9. 🎯 Skill auto-invoke

Como: Smart Router v2 detecta intent → matches 10 patterns **Catálogo:** 779 skills em `skills/` **E2E**

testado: "Pesquise no reddit" → ativa `agent-reach` , "deploy" → ativa `deploy-to-vercel`

10. 🔄 Self-healing (15 fixers)

Como: Detecta erros → aplica fix → loga em `heal_log.json` **Engine:**

`alberto/runtime/self_healing.py` **Status:** AUTO-APLICAÇÃO é raro; a maioria só loga

11. 🌐 3D Web UI

Como: FastAPI server + frontend 3D **Porta:** 8741 (default) **Endpoints:** 30+ (status, chat SSE, memory, squad, security) **Start:** `alberto-serve --port 8741`

12. 🛡️ Defense-in-depth (v1.7)

- `tool_file_write` checado por NemoClaw
- `tool_run_shell` com whitelist de 10+ padrões perigosos

- "Comando:" interceptor validado
- Audit log persistente em `~/.alberto/audit.log`



Capabilities que PRECISAM de credenciais/config

Capability	O que precisa
Voice TTS/STT	Magpie OU ElevenLabs API key
Discord/Telegram/Slack/etc	Bot tokens por plataforma
HomeAssistant	HA instance rodando
xAI search	xAI API key
Email/SMS	SMTP/credentials
All 22 communication tools	Credenciais específicas



O que AIOX é (ainda stub) — confirmação direta

Você perguntou se AIOX ainda é stub. **Sim, é.**

```
$ find upstream/aiox -name "*.py" | wc -l
0          # ZERO arquivos Python

$ find upstream/aiox -name "*.yaml" | wc -l
21         # Só YAMLS de docs

$ npm view @aiox/cli
npm error 404 - '@aiox/cli' is not in this registry.
```

O que isso significa:

1. **AIOX upstream** = só docs + 21 YAMLS descrevendo workflows
2. **Não existe runtime** — nem Python, nem JS CLI, nem nada executável
3. **npm package** `@aiox/cli` **não existe** (já testei no registry)
4. **3 workflows AIOX** em `claude-code-mastery` são specs de paper, não código

Como Alberto usa "AIOX":

```
# alberto/runtime/integration.py
def run_aiox_workflow(name, alberto, input_data=None):
    """Execute an AIOX workflow by name.

    Real AIOX runs via the aiox CLI; for now we parse the YAML and
    execute the steps in sequence using existing Alberto tools.
    """
```

Tradução: Alberto lê o YAML do AIOX e **executa usando o Alberto runner**. Os workflows AIOX são specs — Alberto os interpreta e roda.

Em uso real: você só vê isso na squad `claude-code-mastery` (built-in, descoberta via `alberto squad_list`).

Resumo honesto

Alberto FAZ (real, testado, com LLM real)

- ✓ Chat PT-BR / EN
- ✓ 15 squads multi-persona
- ✓ Vision (2 modelos)
- ✓ Image gen (flux.1-dev)
- ✓ Web search (browser fallback)
- ✓ Code execution (sandboxed)
- ✓ Memory (FTS5)
- ✓ Security scanning (36 padrões)
- ✓ Skill auto-invoke (10 padrões)
- ✓ 3D Web UI
- ✓ 324 testes passing

Alberto NÃO FAZ (sem credenciais externas)

- ✗ Voice TTS/STT real
- ✗ Discord/Telegram/Slack/etc (sem tokens)
- ✗ Real email/SMS
- ✗ Real AIOX CLI (não existe upstream)
- ✗ Real NemoClaw Docker sandbox (sem Docker daemon)

Alberto USA como STUB/REFERÊNCIA

- ⚠ **AIOX** — só YAMLS, não runtime
- ⚠ **NemoClaw full plugin** — só 6/36 features portadas

🎯 Para que serve AGORA

Use cases que FUNCIONAM (com LLM key)

1. **Consultor técnico em PT-BR** — chat direto + squads engineering/content
2. **Automação de pesquisa web** — sem API key, via browser
3. **Geração de imagens criativas** — flux.1-dev com key certa
4. **Code review automatizado** — squad engineering
5. **Triagem de incidentes** — squad incident
6. **Brainstorm multi-perspectiva** — qualquer squad
7. **Security audit de texto** — 36 secret patterns
8. **Bootstrapping de apps** — `alberto app` (smoke level)
9. **Pesquisa acadêmica** — squad research
10. **Hire/interview prep** — squad hiring

Use cases que EXIGEM setup adicional

- **Atendimento ao cliente Discord/Telegram** — você precisa criar bot e dar token
- **TTS/STT para accessibility** — você precisa Magpie OU ElevenLabs
- **Sandbox Docker real** — você precisa Docker daemon rodando
- **Produção multi-tenant** — você precisa k8s + escala